Smart Ambulance IoT

Implementation Choices — CoAP / Californium

Giuseppe Di Blasi
Università degli Studi di Parma

July 3, 2026

Agenda

1. Introduction
2. Architecture
3. Implementation Choices
4. Conclusions

The Smart Ambulance Scenario

- An ambulance $\mathcal{A}$ with two operating environments: **Cockpit** ($\mathcal{A}_C$) and **Rear Cabin** ($\mathcal{A}_R$)
- Two first-aid workers acting as CoAP clients: **Driver** and **Reliever**
- A **GPS** receiver, also modeled as a CoAP client
- Each environment is a CoAP server hosting three resources: temperature sensor, air-conditioning controller, HMI

Focus of this talk

Not the CoAP theory itself, but the concrete **implementation choices** made to satisfy the project requirements.

Two CoAP Servers, Three CoAP Clients

Servers

- `Cockpit` → port 5684
- `RearCabin` → port 5683
- Both host the same three resources: `TemperatureSensor`, `AirConditioningController`, `HMI`

Clients

- `Driver`, `Reliever`, `GPS`
- All extend a common abstract class `SmartAmbulanceClient`
- Shared reusable GET/PUT operations toward HMI, sensor and controller

Design choice

Two independent `CoapServer` instances instead of one server with parametrized resources — keeps the two environments fully decoupled.

Resources as POJO Classes

- Each resource extends `CoapResource` and keeps its state in **private fields**
- State is exposed only via `handleGET` / `handlePUT` overrides — as requested in the project's "General Suggestions"
- No module ever touches another class's internal variables directly

Resources implemented

- `TemperatureSensor` — GET current temperature; observable
- `AirConditioningController` — GET target temp, PUT with permission check
- `HMI` — GET emergency state, PUT with cross-environment propagation

Message Format: Hand-Built JSON

- Payloads built with `String.format`, no external library
- Parsed with a small helper: `extractJsonValue(json, key)`
- Every payload carries a `"Client"` field identifying the sender

```
{
  "Client": "DRIVER",
  "temp_target": 21
}
```

```
extractJsonValue(
  payload, "temp_target")
```

Why it matters

The `Client` field is what the server uses to route authorization logic (next slide).

Authorization Rules on the AC Controller

Driver

- Cockpit → allowed
- Rear Cabin → allowed

Reliever

- Cockpit → FORBIDDEN
- Rear Cabin → allowed

```
if (client.equals("RELIEVER") && port == COCKPIT_PORT) {
    exchange.respond(ResponseCode.FORBIDDEN);
    return;
}
```

Note

The check is done inside `handlePUT`, comparing the `Client` field with the server's own port — no range/value validation is performed on the payload itself.

Dynamic Temperature + Observe Pattern

Gradual convergence

- `TimerTask` with `scheduleAtFixedRate`
- Current temperature moves ± 1°C every 1.5s toward the target
- Triggered by `setTargetTemperature()`

CoAP Observe

- `setObservable(true)`, CON notifications
- Every change calls `changed()`
- Reliever subscribes with `client.observe(...)`
- Unsubscribes with `proactiveCancel()`

Design choice

Simulating a realistic air-conditioning response instead of an instantaneous jump to the target temperature.

HMI Synchronization Between Environments

- When Driver or Reliever update emergency level / location, the `HMI` resource itself opens an internal `CoapClient`
- It sends a PUT to the twin HMI on the other server — a **server-to-server** propagation, not client-to-client

```
updateOtherHMI(): PUT to twin port, payload marked "HMI_UP"
```

Scope of the synchronization

GPS coordinates remain local to the HMI that receives them directly from the GPS client — they are **not** propagated to the other HMI.

End-to-End Simulation

`SmartAmbulanceSimulation.main` orchestrates a scripted scenario, with `Thread.sleep` pauses between steps:

1. Driver sets the temperature in Rear Cabin
2. GPS sends coordinates to the HMI
3. Driver updates emergency/location (propagated to the other HMI)
4. Reliever attempts the same PUT in Cockpit $\rightarrow$ FORBIDDEN
5. Driver switches server and repeats GET/PUT in Cockpit
6. Reliever observes the temperature sensor, then cancels

Summary and Future Extensions

What was implemented

- 2 servers + 3 clients over CoAP/Californium
- Custom JSON with a `Client` field for authorization
- Dynamic temperature via `TimerTask`
- Observe pattern on the sensor
- Server-to-server HMI synchronization

Possible extensions

- Real JSON library (e.g. Gson) instead of manual parsing
- Input validation with `BAD_REQUEST` for out-of-range values
- Transport-layer security (DTLS)
- Persistence of historical data

Thank you for your attention!

Questions & Contacts

- Project: `SmartAmbulanceIoT` — Matricola 386310
- Framework: Java + Californium (CoAP)

Happy to answer any questions.